Labsheet 10 SECS05-AS010

Mastery in Data Analytics and Visualizations and Career Advancement

Diksha Saini CU240251581

```
import kagglehub
path = kagglehub.dataset_download("pavansubhasht/ibm-hr-analytics-attrition-dataset")
```

```
Using Colab cache for faster access to the 'ibm-hr-analytics-attrition-dataset' dataset.
```

```
# 1. import the dataset, then perform data preprocessing
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(f'{path}/WA_Fn-UseC_-HR-Employee-Attrition.csv')
print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (1470, 35)
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	Emplo
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	

5 rows × 35 columns

```
# 2. Data Preprocessing & Cleaning
print("Null values count:\n", df.isnull().sum().sum())
print("Duplicate rows count:", df.duplicated().sum())

# Drop non-informative constant or ID columns
drop_cols = ['EmployeeCount', 'EmployeeNumber', 'Over18', 'StandardHours']
df.drop(columns=drop_cols, inplace=True)

# Convert binary target variable 'Attrition' to numeric (Yes: 1, No: 0)
df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})
```

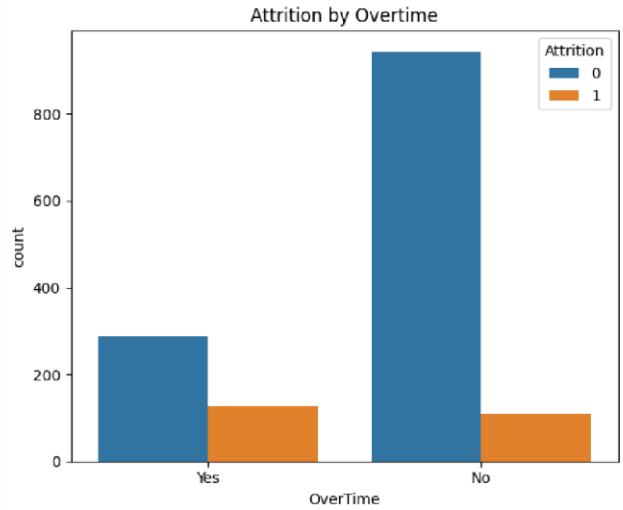
```
Null values count:
 0
Duplicate rows count: 0
```

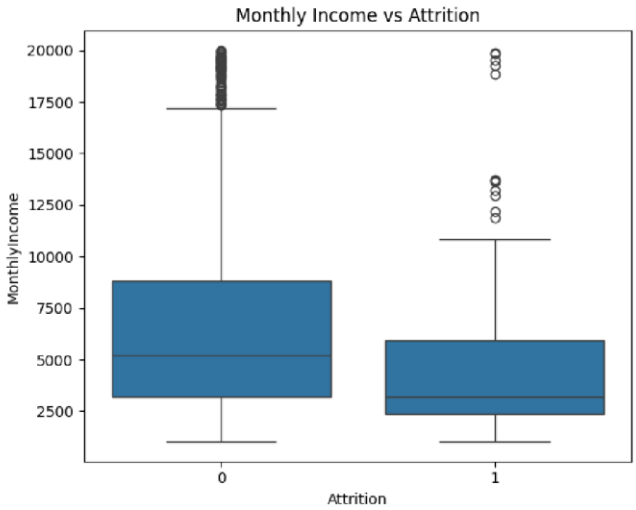
```
# 3. Exploratory Data Analysis
print("Attrition Distribution:\n", df['Attrition'].value_counts(normalize=True))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)

sns.countplot(x='OverTime', hue='Attrition', data=df)
plt.title('Attrition by Overtime')
plt.subplot(1, 2, 2)

sns.boxplot(x='Attrition', y='MonthlyIncome', data=df)
plt.title('Monthly Income vs Attrition')
plt.tight_layout()
plt.show()
```

```
Attrition Distribution:
 Attrition
0    0.838776
1    0.161224
Name: proportion, dtype: float64
```






```
# 4. Statistical Analysis & Hypothesis Testing
from scipy import stats
income_attrition = df[df['Attrition'] == 1]['MonthlyIncome']
income_no_attrition = df[df['Attrition'] == 0]['MonthlyIncome']
t_stat, p_val = stats.ttest_ind(income_attrition, income_no_attrition, equal_var=False)
print(f"T-statistic: {t_stat:.4f}, p-value: {p_val:.4e}")
if p_val < 0.05:
    print("Reject Null Hypothesis: Difference in Monthly Income is statistically significant.")
else:
    print("Fail to reject Null Hypothesis.")
```

```
T-statistic: -7.4826, p-value: 4.4336e-13
Reject Null Hypothesis: Difference in Monthly Income is statistically significant.
```

```
# 5. Feature Engineering & Data Preparation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Separate features and target
X = df.drop(columns=['Attrition'])
y = df['Attrition']

# One-Hot Encoding for remaining categorical columns
X = pd.get_dummies(X, drop_first=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# 6. Predictive Modeling & Model Building
from sklearn.ensemble import RandomForestClassifier

# Build model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predictions
y_pred = rf_model.predict(X_test)
y_pred_proba = rf_model.predict_proba(X_test)[:, 1]
```

```
# 7. Model Evaluation
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print(f"ROC-AUC Score: {roc_auc_score(y_test, y_pred_proba):.4f}")
```

```
Confusion Matrix:
 [[240   7]
 [ 42   5]]

Classification Report:
              precision    recall  f1-score   support

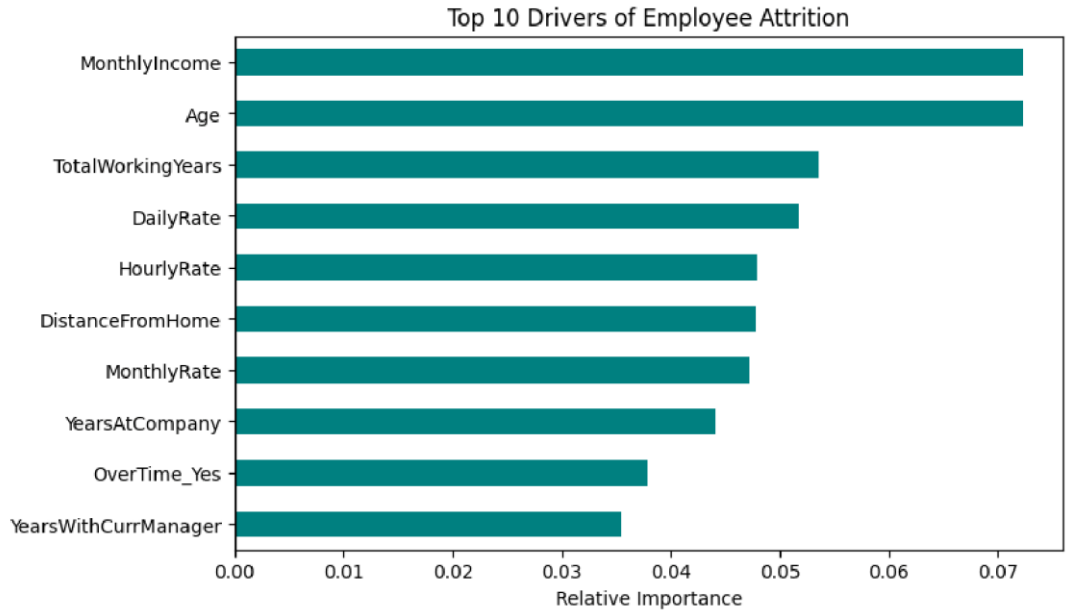
           0       0.85      0.97      0.91       247
           1       0.42      0.11      0.17        47

    accuracy                           0.83       294
   macro avg       0.63      0.54      0.54       294
weighted avg       0.78      0.83      0.79       294

ROC-AUC Score: 0.7862
```

```
# 8. Extract Feature Importance & Key Insights
importances = pd.Series(rf_model.feature_importances_, index=X.columns)
top_10 = importances.nlargest(10)

plt.figure(figsize=(8, 5))
top_10.plot(kind='barh', color='teal')
plt.title('Top 10 Drivers of Employee Attrition')
plt.gca().invert_yaxis()
plt.xlabel('Relative Importance')
plt.show()
```



9.Business Intelligence Dashboard Integration: To convert this analysis into an executive BI dashboard:

1. Export df to CSV (df.to_csv('attrition_processed.csv', index=False)).
2. Import into Power BI or Tableau.
3. Build 3 core visual sections:

- KPI Cards: Total Employees, Overall Attrition Rate, Average Monthly Income, Average Years at Company.
- Risk Heatmaps: Attrition Rate by Department vs Overtime status.
- Driver Decomposition: Attrition Rate mapped against Distance From Home and Stock Option Level.

10.Actionable Business Recommendations: Formulate strategies based on insights from statistical testing and model feature importances:

- Target Overtime Burnout: Employees working overtime show a significantly higher rate of attrition. Establish thresholds for forced overtime and offer compensatory rest/bonuses.
- Review Entry-Level Compensation: Monthly income was identified as a primary statistical driver of turnover. Conduct market-rate adjustments for low-tenure roles.
- Career Growth Pathways: Focus retention budgets on mid-level employees showing low promotion frequency over 3+ years.